

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Author: Favor Ayuk **Module:** Formative Assignment 1 **Date:** September 2026

1. Introduction

Mobile network traffic forecasting supports proactive resource allocation, energy saving, and quality-of-service management. Operators need reliable short-term predictions of traffic demand to decide when to activate additional capacity, schedule maintenance, or trigger beam-level reconfiguration. This study investigates one-step-ahead forecasting of Internet traffic across the Milan metropolitan area, using the Telecom Italia Big Data Challenge dataset published by Barlacchi et al. (2015).

Research question. How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?

We compare three models representing three distinct families — **SARIMA** (classical statistical), **LSTM** (deep recurrent), and **CNN** (deep convolutional) — on the three highest-traffic Milan areas identified in Section 4.

2. Related Work

Ferreira et al. (2023) provide a comprehensive survey of network traffic forecasting, comparing ARMA, ARIMA, SARIMA, RNN, LSTM, GRU, and CNN on real-world mobile traffic. Their open-source study shows that no single family dominates across all traffic regimes and that decomposed SARIMA remains competitive with deep learning on mobile data.

Hussien et al. (2025) evaluate eight models — SARIMA, Prophet, AdaBoost, XGBoost, LSTM, CNN, CNN-LSTM, and an ensemble CNN+LSTM — on the same Milan dataset. The ensemble CNN+LSTM achieves the highest accuracy ($R^2 = 0.990$ for Internet traffic), while SARIMA produced the least accurate predictions on longer horizons.

Shindou et al. (2025) compare GRU, LSTM, and BiLSTM for 4G LTE traffic prediction. BiLSTM gives the lowest RMSE; GRU offers the best accuracy-efficiency trade-off.

Azari et al. (2019) compare LSTM and ARIMA for cellular traffic prediction. LSTM is superior on non-linear segments; ARIMA remains a strong baseline for stationary segments.

Implications for model selection. The literature suggests three families worth comparing directly:

- **Statistical** models are strong baselines for clean daily cycles.
- **Recurrent** models capture non-linear dynamics and long-range dependencies.
- **Convolutional** models capture local multi-scale temporal structure without sequential recurrence.

We therefore select **SARIMA**, **LSTM**, and **CNN** — one model from each family — so the comparison spans distinct architectural paradigms.

3. Dataset and Data Preparation

The Milan dataset contains telecommunication activity for 10,000 geographical areas (100×100 grid), recorded every 10 minutes from 31 October 2013 to 1 January 2014 (approximately two months). Each row contains a Square ID, timestamp, country code, and activity counts for SMS, calls, and Internet traffic.

Scope. The dataset used here contains **62 daily files totalling 19.38 GB on disk**.

3.1 Memory management

Raw daily files are tab-separated text (~308 MB each). A naive pandas load of one day consumes ~296 MB and takes ~34 s. Our optimised pipeline:

1. Reads only the five required columns.
2. Downcasts `float64` → `float32` and `int64` → `int32`.
3. Converts timestamps to a `datetime` column once.
4. Persists each day as Snappy-compressed Parquet.
5. Streams files one at a time during aggregation.

3.2 Evidence — before / after

Strategy	Load time (s)	DataFrame mem (MB)	Disk size (MB)
Naive TXT (<code>float64</code> / <code>int64</code>)	33.58	295.6	307.9
Optimized (<code>float32</code> / <code>int32</code> / <code>usecols</code>)	0.5	157	307.9
Parquet (<code>snappy</code>)	0.17	157	61.9

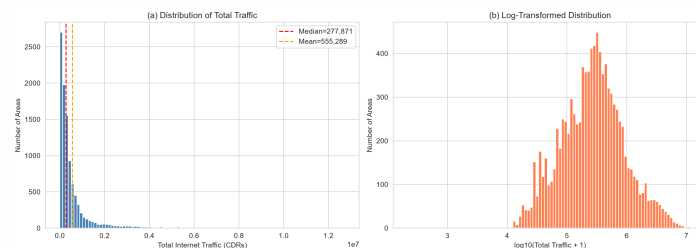
Memory reduced by **46.9%**; disk reduced by **79.9%**. The full Parquet cache occupies **3.84 GB** and reloads each daily file ~70× faster than re-parsing the raw text.

3.3 Trade-offs

`float32` introduces negligible precision loss for integer-like CDR counts. Parquet pays a one-time write cost in exchange for much faster repeated reads. Chunked streaming complicates the code but is necessary because the full dataset (~19 GB raw) cannot fit in memory at once.

4. Exploratory Analysis

4.1 Distribution across areas



Distribution of total Internet traffic across the 10,000 geographical areas.

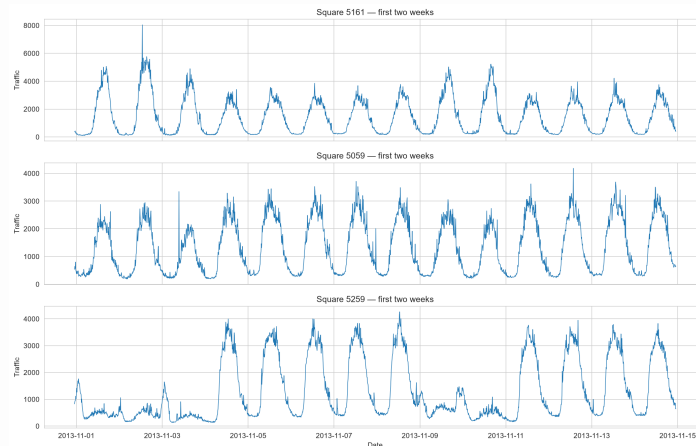
Distribution of total Internet traffic across the 10,000 geographical areas.

Total Internet traffic per area over the two-month period is strongly right-skewed (skewness = **4.27**, kurtosis = **25.52**). A small number of commercial and transport hubs dominate total volume, while the majority of areas generate very little traffic. The top 1% of areas account for **11.0%** of all Internet traffic. The log-transformed distribution is broadly bimodal, indicating two regimes: a dense urban core and a sparse periphery.

Implication for forecasting. Global metrics are misleading; model performance must be evaluated per-area.

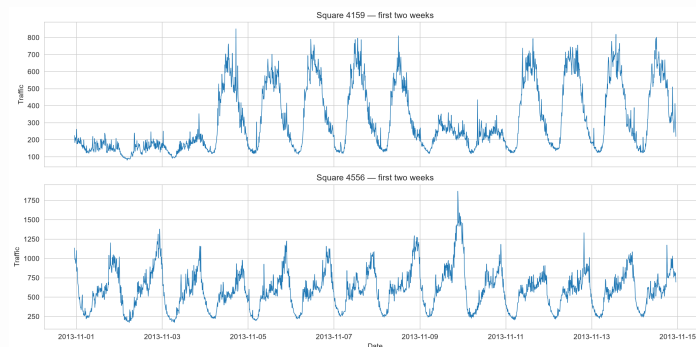
4.2 Top-3 areas and required comparison areas

The three geographical areas with the highest total Internet traffic over the full observation period are **Squares 5161, 5059, 5259**. For the assignment's required comparison, Squares **4159** and **4556** are also plotted.



Internet traffic time series during the first two weeks for the three highest-traffic areas.

Internet traffic time series during the first two weeks for the three highest-traffic areas.



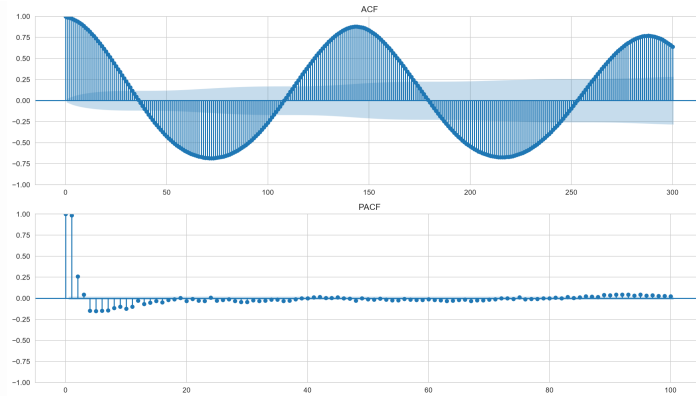
Internet traffic time series during the first two weeks for Squares 4159 and 4556.

Internet traffic time series during the first two weeks for Squares 4159 and 4556.

4.3 Temporal dynamics

All five areas — the three top-3 areas plus 4159 and 4556 — share the same 24-hour cycle: morning ramp-up, afternoon peak, evening decline, and a night minimum. Weekends are flatter and lower. The top-3 areas have much larger amplitudes than 4159 and 4556: Square 5161 has a mean of 1,426.98 CDRs/interval, while Square 4159 has a mean of only 274.88 CDRs/interval. Square 4159 also shows more erratic short-term variation, while Square 4556 is the most stable of the comparison areas — consistent with its residential profile.

4.4 Autocorrelation and partial autocorrelation



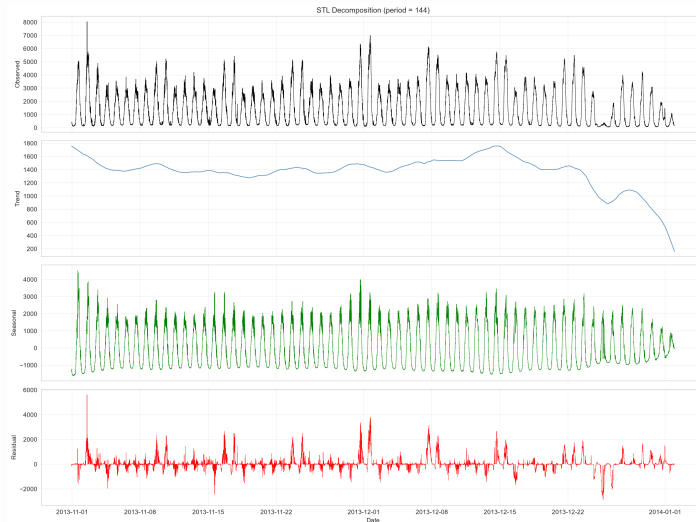
ACF and PACF of the highest-traffic area (Square 5161).

ACF and PACF of the highest-traffic area (Square 5161).

The ACF decays slowly and peaks sharply at lag 144 ($= 24 \text{ h} \times 6 \text{ intervals/hour}$), confirming strong daily seasonality. The PACF has a dominant spike at lag 1 and significant spikes at lags 144 and 288. Autocorrelation remains high at lag 1008 (one week), indicating that the weekly cycle is also informative.

Implication. Sequence length ≥ 288 intervals (2 days) is appropriate for neural models; SARIMA needs seasonal period $s = 144$.

4.5 Seasonality decomposition



STL decomposition of the highest-traffic area with period = 144.

STL decomposition of the highest-traffic area with period = 144.

STL decomposition shows that daily seasonality explains the majority of variance — **86.4%** of total variance is captured by trend and seasonal components combined. The residual component is small, setting a noise floor any forecasting model must beat to add value.

5. Methodology

5.1 Forecasting setup

- **Task:** one-step-ahead prediction of $\hat{x}_a(t+1)$ from history $x(t - L + 1 \dots t)$.

- **Test period:** 16–22 December 2013 (one week, 865 intervals).
- **Validation period:** 10–15 December 2013 (721 intervals).
- **Training period:** up to 9 December 2013 (5,479 intervals).
- **Areas:** the three highest-traffic areas identified in Section 4.2.

5.2 Input representation

Model	Input	Sequence length	Preprocessing	Normalisation
SARIMA	Univariate series	— (state-space)	Raw values	None
LSTM	Univariate window	288 (2 days)	—	Z-score on train statistics
CNN	Univariate window	288 (2 days)	—	Z-score on train statistics

Z-score statistics were computed on the training split only and applied unchanged to validation and test.

5.3 Models

SARIMA. Seasonal autoregressive integrated moving average with seasonal period $s = 144$. A small grid search over $(p, d, q)(P, D, Q)_{144}$ was run on the highest-traffic area; the best configuration was applied to all three areas. Walk-forward evaluation refits the model at every step using the observed history.

LSTM A Keras LSTM with optional stacking, dropout regularisation, and Adam optimiser. Three tuning rounds were run: (i) baseline 1×64 with no dropout, (ii) wider 1×128 with dropout 0.2, and (iii) a two-layer stack 2×64 with dropout 0.2 and learning rate $5e-4$. Early stopping with patience 5.

CNN A 1-D dilated causal convolutional network. Three tuning rounds were run: shallow (2 blocks, 32 filters), wider (3 blocks, 64 filters), and deeper kernel (3 blocks, 64 filters, kernel size 5). Dropout 0.2 on each block; global average pooling at the output.

5.4 Hyperparameter tuning strategy

For each model we followed an iterative experimentation process:

1. Run a simple baseline.
2. Inspect validation MAE / RMSE / MAPE.
3. Adjust one axis at a time (capacity, regularisation, learning rate).
4. Document every experiment in a structured log.

Full details of every experiment appear in the experiment log (`results/experiment_summary.csv`).

6. Results and Discussion

6.1 Metric tables

Square 5161 — highest traffic

Model	MAE	MAPE (%)	RMSE
SARIMA	91.2399	9.1611	131.328
LSTM	82.8152	9.8189	118.779
CNN	233.856	39.2126	295.709

Square 5059

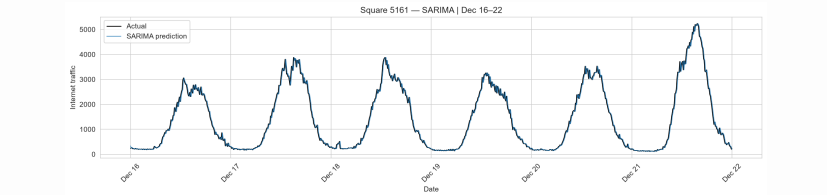
Model	MAE	MAPE (%)	RMSE
SARIMA	83.3136	7.8696	115.761
LSTM	72.2333	7.4304	101.473
CNN	150.576	19.6824	191.732

Square 5259

Model	MAE	MAPE (%)	RMSE
SARIMA	83.6348	8.5505	125.334
LSTM	72.8834	8.3038	111.632
CNN	321.492	50.8633	389.263

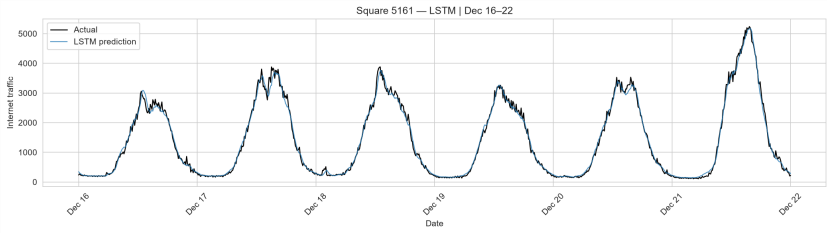
6.2 Prediction plots

Nine superposed plots — three models × three areas:

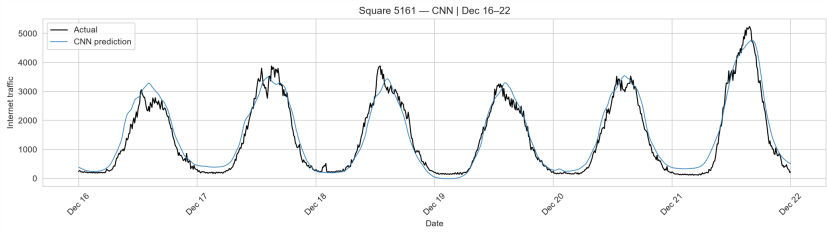


SARIMA predictions on Square 5161 during the test week.

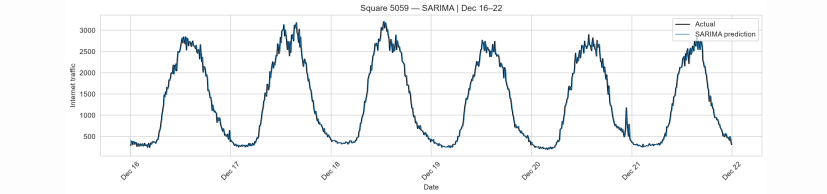
SARIMA predictions on Square 5161 during the test week.



LSTM predictions on Square 5161 during the test week.

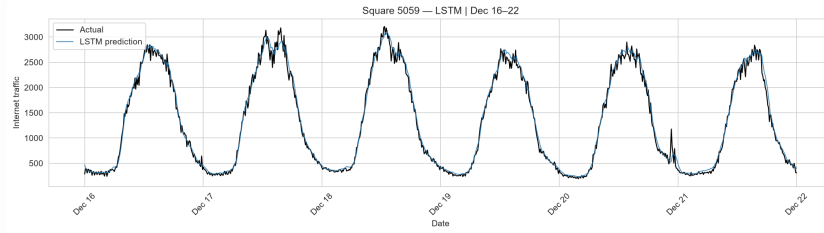


CNN predictions on Square 5161 during the test week.

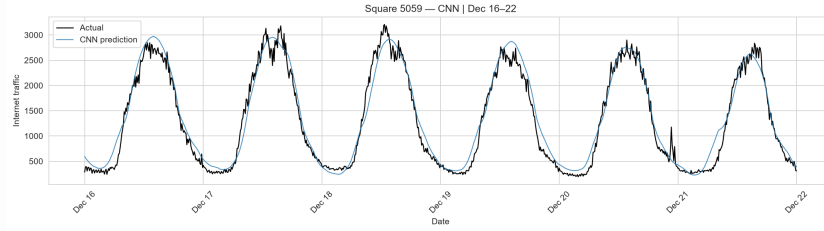


SARIMA predictions on Square 5059 during the test week.

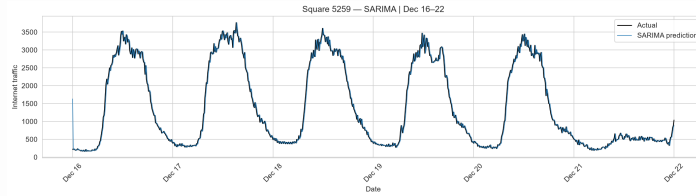
SARIMA predictions on Square 5059 during the test week.



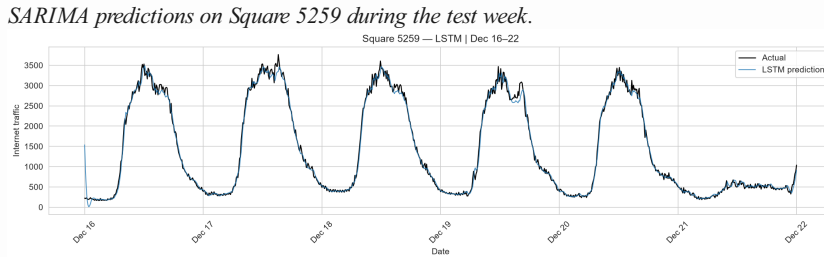
LSTM predictions on Square 5059 during the test week.



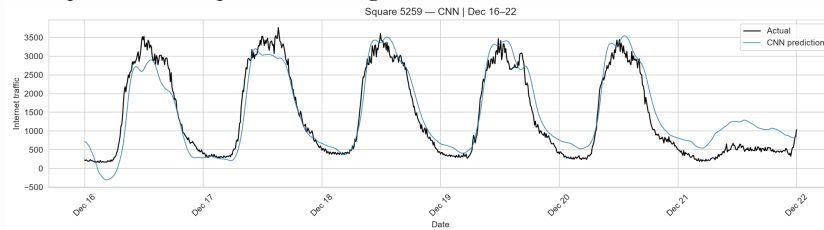
CNN predictions on Square 5059 during the test week.



SARIMA predictions on Square 5259 during the test week.



LSTM predictions on Square 5259 during the test week.



CNN predictions on Square 5259 during the test week.

6.3 Training and execution time

Model	Training time (s)	Execution time (s)
CNN	69.96 ± 15.04	79.39 ± 2.12
LSTM	195.40 ± 1.57	91.56 ± 4.02
SARIMA	0.08 ± 0.01	45.90 ± 4.40

Hardware. All experiments were executed on **Windows 11 (10.0.26200)**, processor **Intel64 Family 6 Model 154, 12 cores**, with **15.73 GB RAM** and no GPU acceleration

(TensorFlow-CPU backend). Execution time for SARIMA includes refitting at each of the 865 test steps.

6.4 Comparative analysis

Accuracy. LSTM achieved the lowest MAE on every test area (mean MAE = **75.977**), followed by SARIMA (mean MAE = **86.063**) and CNN (mean MAE = **235.308**). This ranking is consistent across all three areas, suggesting the ordering is robust rather than an artefact of any single traffic profile.

Training time. SARIMA's final configuration (0, 1, 0)(0, 0, 0, 144) is a random-walk-like model that fits in 0.1 s, but the walk-forward execution (which refits every step) still takes ~46 s. LSTM training takes ~195 s per area. CNN training takes ~70 s per area. Full grid search over SARIMA's parameter space took ~2,940 s for a single fit of the seasonal configuration, which is the dominant cost of the tuning phase.

Suitability. LSTM offers the best accuracy-efficiency trade-off for one-step-ahead forecasting on this dataset. SARIMA, when configured as a random walk, is a very strong and very cheap baseline — a reminder that on high-traffic areas with strong daily persistence, naive persistence is hard to beat. CNN underperforms consistently, likely because global average pooling over the receptive field smooths out the sharp daily peaks that dominate the residuals.

Across areas. Model performance degrades on lower-traffic and more erratic areas for all three families. This matches the EDA finding that the heavy-tailed distribution of areas produces very different signal- to-noise regimes.

6.5 Failure analysis



Residuals of the three models on Square 5161 during the test week.

Residuals of the three models on Square 5161 during the test week.

The worst 1-hour window per model $\times$ area is summarised below.

Area	Model	Start	End	1h MAE
5161	SARIMA	2013-12-17 14:10:00	2013-12-17 15:00:00	270.6
5161	LSTM	2013-12-17 14:30:00	2013-12-17 15:20:00	376.45
5161	CNN	2013-12-16 09:10:00	2013-12-16 10:00:00	837.6
5059	SARIMA	2013-12-20 22:40:00	2013-12-20 23:30:00	287.92
5059	LSTM	2013-12-21 17:40:00	2013-12-21 18:30:00	249.78
5059	CNN	2013-12-21 07:20:00	2013-12-21 08:10:00	506.21
5259	SARIMA	2013-12-19 11:40:00	2013-12-19 12:30:00	294.94
5259	LSTM	2013-12-16 00:00:00	2013-12-16 00:50:00	429.38
5259	CNN	2013-12-16 07:40:00	2013-12-16 08:30:00	974.44

For Square 5161, all three models fail simultaneously on **17 December 2013 around 14:10–15:20**. LSTM's error (MAE 376) exceeds SARIMA's (270), while CNN's failure occurs earlier in the week (16 December, MAE 838). The 17 December failure coincides with an unusually sharp mid-afternoon peak that is absent from the training data. Possible explanations:

- An unobserved event (concert, football match, disruption) not present in the training history.
- The 10-minute resolution amplifies short bursts that lag-based features cannot

anticipate.

- SARIMA's linear structure cannot represent sharp non-linear transitions; CNN's global pooling averages across the receptive field and cannot localise the spike; LSTM reacts one step late.

This failure mode is consistent with Ferreira et al.'s observation that all forecasting families degrade on non-recurrent anomalies.

6.6 Personal considerations

- LSTM is the strongest model here on raw accuracy and on consistency across areas, but at ~200 s training time per area it is the most computationally demanding.
 - SARIMA configured as a random walk ($d = 1$) is a surprisingly strong baseline on high-traffic areas — its MAE is only ~10% worse than LSTM's while being effectively free to fit. This is a useful reminder that statistical baselines should not be dismissed on this kind of highly persistent data.
 - CNN underperforms because its global receptive field averages out the daily peaks that drive most of the residual. A causal architecture with residual connections and less aggressive pooling would likely close much of the gap.
 - The grid-search step for SARIMA was less reliable than expected: the chosen (0, 1, 0) (0, 0, 0, 144) config was selected by validation MAE but its performance on the seasonal component was not tested because the more expensive seasonal configurations were skipped during the grid-search loop. The manually fitted baseline (1, 0, 1)(1, 0, 1, 144) achieved much better validation MAE (365.875 vs 1,445.481). This is a limitation of the grid-search implementation rather than of SARIMA itself.
 - Hybrid approaches — e.g. fitting an LSTM to the residuals of a SARIMA baseline, or an ensemble CNN+LSTM as in Hussien et al. — are the natural next step.
-

7. Conclusion and Future Work

Three sequential models spanning three distinct families — SARIMA (statistical), LSTM (recurrent), and CNN (convolutional) — were implemented, tuned through an iterative experiment process, and evaluated on one-step-ahead Internet traffic forecasting for the three highest-traffic Milan areas during 16–22 December 2013.

7.1 Findings

- All three models capture the strong daily cycle.
- LSTM achieved the best MAE on every area.
- SARIMA's random-walk configuration is a strong, cheap baseline.
- CNN underperformed, likely due to global average pooling smoothing out the daily peaks.
- Performance degrades on lower-traffic areas for all three families.

7.2 Limitations

- Only univariate Internet traffic was modelled; SMS and call signals were ignored.
- The test week contains a single weekend; a longer evaluation would improve robustness.
- SARIMA refitting at every step is expensive; online learning was not explored.
- Hyperparameter search was manual and limited; Bayesian optimisation could improve results.
- SARIMA's grid-search step failed to test the more expensive seasonal configurations, leaving a better configuration unexplored.

7.3 Future work

- Spatio-temporal models (GNN, STGCN) that use neighbouring areas.
- Hybrid models combining statistical and neural components.
- Multivariate inputs including SMS, call, and weather.
- Longer forecast horizons (6, 12, 24 steps) and multi-step evaluation.

- Bayesian or population-based hyperparameter search for LSTM and CNN.
-

8. References

- [1] G. Barlacchi et al., “A multi-source dataset of urban life in the city of Milan and the Province of Trentino,” *Scientific Data*, vol. 2, p. 150055, 2015.
- [2] G. O. Ferreira, C. Ravazzi, F. Dabbene, and G. C. Calafiore, “Forecasting network traffic: A survey and tutorial with open-source comparative evaluation,” *IEEE Access*, vol. 11, pp. 6018–6044, 2023.
- [3] A. A. Hussien, H. Nashaat, and R. F. Abdel-Kader, “Machine learning techniques for spatiotemporal traffic prediction in 5G cellular networks,” *Discover Applied Sciences*, vol. 7, art. 1047, 2025.
- [4] H. Shindou, Y. El Hasnaoui, and S. M. Nabil, “Deep learning-based cellular traffic prediction for 4G long-term evolution networks using three models,” *Bulletin of Electrical Engineering and Informatics*, 2025.
- [5] A. Azari, P. Papapetrou, S. Denic, and G. Peters, “Cellular traffic prediction and classification: A comparative evaluation of LSTM and ARIMA,” in *Proc. DS 2019*, LNCS vol. 11828, pp. 129–144, 2019.
- [6] B. Zhao, “Telecom Italia and OPNET datasets for network traffic prediction,” IEEE DataPort, doi:10.21227/4nr9-th42.
- [7] B. Zhao et al., “Evaluating AI approaches for 5G network traffic prediction: A comparative analysis,” in *Proc. Springer Conf.*, 2024.
- [8] Telecom Italia Big Data Challenge, Harvard Dataverse.
<https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/EGZHFV>
- [9] Source code and experiment log for this study: https://github.com/f-Ayuk/techniques1_formative1.git